

Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator

Haocong Luo¹, Yahya Can Tuğrul², F. Nisa Bostancı³, Ataberk Olgun⁴, *Graduate Student Member, IEEE*,
A. Giray Yağlıkcı⁵, *Graduate Student Member, IEEE*, and Onur Mutlu⁶, *Fellow, IEEE*

Abstract—We present Ramulator 2.0, a highly modular and extensible DRAM simulator that enables rapid and agile implementation and evaluation of design changes in the memory controller and DRAM to meet the increasing research effort in improving the performance, security, and reliability of memory systems. Ramulator 2.0 abstracts and models key components in a DRAM-based memory system and their interactions into shared *interfaces* and independent *implementations*. Doing so enables easy modification and extension of the modeled functions of the memory controller and DRAM in Ramulator 2.0. The DRAM specification syntax of Ramulator 2.0 is concise and human-readable, facilitating easy modifications and extensions. Ramulator 2.0 implements a library of reusable templated lambda functions to model the functionalities of DRAM commands to simplify the implementation of new DRAM standards, including DDR5, LPDDR5, HBM3, and GDDR6. We showcase Ramulator 2.0's modularity and extensibility by implementing and evaluating a wide variety of RowHammer mitigation techniques that require *different* memory controller design changes. These techniques are added modularly as separate implementations *without* changing *any* code in the baseline memory controller implementation. Ramulator 2.0 is rigorously validated and maintains a fast simulation speed compared to existing cycle-accurate DRAM simulators.

Index Terms—Computer architecture, computer simulation systems engineering and theory, computers and information processing, DRAM systems engineering and theory, memory architecture computers and information processing, memory management computers and information processing, memory, modeling, random access memory, scalability, simulation, system analysis and design, system simulation systems engineering and theory.

I. INTRODUCTION

CYCLE-ACCURATE DRAM simulators enable modeling and evaluation of detailed operations in the memory controller and the DRAM device. In recent years, growing research and design efforts in improving the performance, security, and reliability of DRAM-based memory systems require a cycle-accurate simulator that facilitates rapid and agile implementation and evaluation of intrusive design changes (i.e., modification of functionalities of the simulated system as opposed to simple

parameter changes) in the memory controller and DRAM. Unfortunately, existing cycle-accurate DRAM simulators are not modular and extensible *enough* to meet such a requirement.

We identify two key issues in the design and implementation of existing cycle-accurate DRAM simulators. First, they do *not* model key components of a DRAM-based memory system in a *fundamentally modular* way, making it difficult to implement and maintain different intrusive design changes. For example, USIMM [1] does not separate the DRAM specification from the memory controller. Similarly, the templated implementations of the DRAM specifications in Ramulator [2] (referred to as Ramulator 1.0 in this paper) cause undesired coupling between the DRAM specification and the memory controller.

Second, existing simulators do *not* implement DRAM specifications in a concise and intuitive way, making it difficult to add new DRAM commands and define new timing constraints. For example, both DRAMsim2 [3] and DRAMsim3 [4] implement a single DRAM device model that aggregates all the DRAM specifications from all supported DRAM standards in a single C++ class. Ramulator 1.0's DRAM specifications are based on low-level and verbose C++ syntax (e.g., it uses eight full lines of C++ code just to specify solely a single `tCCD_L` timing constraint in DDR4 [5]).

To address these issues, we present Ramulator 2.0 [6], a successor to Ramulator 1.0 [2] that provides an easy-to-use, modular, and extensible software infrastructure for rapid and agile implementation and evaluation of DRAM-related research and design ideas. Ramulator 2.0 has two distinguishing features. First, it implements a modular and extensible code framework by identifying and modeling the key components in a DRAM-based memory system into separate *interfaces* and *implementations*. With this framework, different design changes (e.g., different address mapping schemes, request scheduling policies, new DRAM standards, RowHammer mitigations) can be implemented as *independent* implementations that share the *same* interface, enabling easy modification and extension of Ramulator 2.0.

Second, to facilitate easy modification of DRAM specifications (e.g., DRAM organization, commands, timing constraints), Ramulator 2.0 implements concise and human-readable definitions of DRAM specifications on top of the lookup table based hierarchical DRAM device model in Ramulator 1.0. Ramulator 2.0's DRAM specifications 1) are defined with simple string literals, 2) leverage permutations of different DRAM commands to concisely define timing constraints, and 3) use a library of templated lambda functions that are *reusable* across different DRAM standards to define the functionalities of DRAM commands (e.g., the same RFM command implementation can be (and is) used by DDR5 [7], LPDDR5 [8], and GDDR6 [9], HBM3 [10]). These improvements are implemented with the new features of C++20 [11] (e.g., constant-evaluated immediate

Manuscript received 21 August 2023; accepted 9 October 2023. Date of publication 17 November 2023; date of current version 3 May 2024. This work was supported in part by Google, Huawei, Intel, Microsoft, VMware, SRC, EFCL, and in part by the InnoHK ACCESS Center. (Corresponding author: Haocong Luo.)

The authors are with the ETH Zürich, 8092 Zürich, Switzerland (e-mail: richardluo723@gmail.com; yahyacantugrul@gmail.com; nisabstnc@gmail.com; olgunataberk@gmail.com; agirayyaglikci@gmail.com; omutlu@gmail.com).

Ramulator 2.0 is open-sourced under the permissive MIT license at <https://github.com/CMU-SAFARI/ramulator2>.

Digital Object Identifier 10.1109/LCA.2023.3333759

functions), enabling significant duplicate-code reduction and easy modification and extension of the modeled DRAM device's functionalities *without* sacrificing simulation speed.

We showcase the modularity and extensibility of Ramulator 2.0 by implementing and evaluating a variety of RowHammer mitigation techniques (PARA [12], TWiCe [13], Graphene [14], Hydra [15], Randomized Row-Swap (RRS) [16], and an ideal refresh-based mitigation [17]) that require *different* additional functionalities in the memory controller. These RowHammer mitigations plug themselves into the *same* baseline memory controller implementation *without* changing the memory controller's code, which was not possible in Ramulator 1.0 [2] and is not possible in any other DRAM simulator we are aware of [1], [3], [4].

In summary, the key features and contributions of Ramulator 2.0 are:

- Ramulator 2.0 is a modular and extensible DRAM simulator written in C++20 [11] that enables rapid and agile implementation and evaluation of design changes in the memory system. Ramulator 2.0 can either work as a standalone simulator, or be used as a memory system library by a system simulator (e.g., gem5 [18], ZSim [19]).
- We showcase the modularity and extensibility of Ramulator 2.0 by implementing and evaluating six different RowHammer mitigation techniques as plugins to a single unmodified memory controller implementation.
- Ramulator 2.0 implements a wide range of new DRAM standards, including DDR5 [7], LPDDR5 [8], HBM3 [10], and GDDR6 [9] (as well as old ones, e.g., DDR3 [20], DDR4 [5], HBM(2) [21]).
- Ramulator 2.0 is rigorously validated and maintains a fast simulation speed compared to existing cycle-accurate DRAM simulators.
- We open-source Ramulator 2.0 [6] under the permissive MIT license to facilitate and encourage open research and agile implementation of new ideas in memory systems. We also integrate it with gem5 [18].

II. RAMULATOR 2.0 DESIGN FEATURES

We walk through the two key design features of Ramulator 2.0 that enable rapid and agile implementation of design changes in the memory system. Section II-A introduces the high-level software architecture of Ramulator 2.0 based on the key concepts of *interface(s)* and *implementation(s)*. Section II-A1 provides a deeper look into the modularity and extensibility enabled by Ramulator 2.0 by showcasing how different RowHammer mitigations can all be implemented as *plugins* of the same baseline unmodified memory controller implementation. Section II-B introduces the concise and human-readable DRAM specification syntax of Ramulator 2.0 that facilitates easy modification and extension of the functionality of the DRAM device.

A. Modular and Extensible Software Architecture

Ramulator 2.0 models all components in a DRAM-based memory system with two fundamental concepts, *Interface* and *Implementation*, to achieve high modularity and extensibility. An interface is an abstract C++ class defined in a .h header file that models the common high-level functionality of a component as seen by other components in the system. An implementation is a concrete C++ class defined in a .cpp file that inherits from an interface, modeling the actual behavior of a component.

Components interact with each other through pointers to each other's interfaces stored in the implementations. With such a design, the functionality of a component can be easily changed by instantiating a different implementation for the same interface, involving *no* changes in the code of unrelated components.

Fig. 1 shows the high-level software architecture of Ramulator 2.0 with the key interfaces we identify in a DRAM-based memory system (dark boxes) and their typical implementations (light boxes) when modeling a DDR5 system with RowHammer mitigation. The arrows illustrate the relationships among different components in the simulated system (i.e., how they call each other's interface functions). We highlight the memory request path with red arrows, DRAM command path with blue arrows, and DRAM maintenance requests (e.g., refreshes) with green arrows. A typical execution of the simulation is as follows: First, memory requests are sent ① from the frontend (either parsed from traces or generated by another simulator, e.g., gem5 [18]) to the memory system, where the memory addresses are mapped ② to the DRAM organization through the address mapper. Then, the requests are enqueued ③ in the request buffers of the DRAM controller. The DRAM controller is responsible for 1) ticking the refresh manager ④, which could enqueue high-priority maintenance requests (e.g., refreshes) back to the controller, 2) querying the request scheduler ⑤, which in turn consults the DRAM device model ⑥ to decode the best DRAM command to issue ⑦ to serve a memory request, and 3) issuing the DRAM command ⑧, which updates the behavior and timing information of the DRAM device model. Finally, the memory controller executes the finished request's callback ⑨ to notify the frontend.

Users can easily extend Ramulator 2.0 without intrusive changes to existing code by creating different implementations of each existing interface in three easy steps: 1) create a new .cpp file, 2) create the new implementation class that inherits from both the implementation base class and the existing interface class, and 3) implement the new functionality in the new implementation class. Similarly, a new interface can be added simply adding a .h file containing the abstract interface class definitions. All interfaces and implementations in Ramulator 2.0 *register themselves* to a class registry that bookkeeps the relationship among different interfaces and implementations. Using this registry, Ramulator 2.0 automatically recognizes and instantiates different implementations for each interface from a human-readable configuration file. Users do *not* need to manually maintain any boilerplate code to describe the relationships between interfaces and implementations.

1) *Memory Controller Plugins*: We make a key observation that many modeled functions in the memory controller (e.g., controller-based RowHammer mitigations that tracks the issued activation commands) and utilities needed for evaluation (e.g., collecting statistics from the issued DRAM commands and analyzing the memory access patterns) are triggered (updated) by the currently-scheduled DRAM command. To avoid having many similar memory controller implementations for every single such modeled function and utility, we model these functions as plugins to the memory controller. As an example, Fig. 2 shows in detail how various RowHammer mitigation techniques (e.g., PARA [12], Graphene [14], Hydra [15]) can be implemented as such controller plugins.

The plugin interface has a simple `update(DRAM_CMD, ADDR)` function that the controller calls (① in Figs. 1 and 2) to notify the plugin implementations about the DRAM command and address issued by the memory controller. The RowHammer

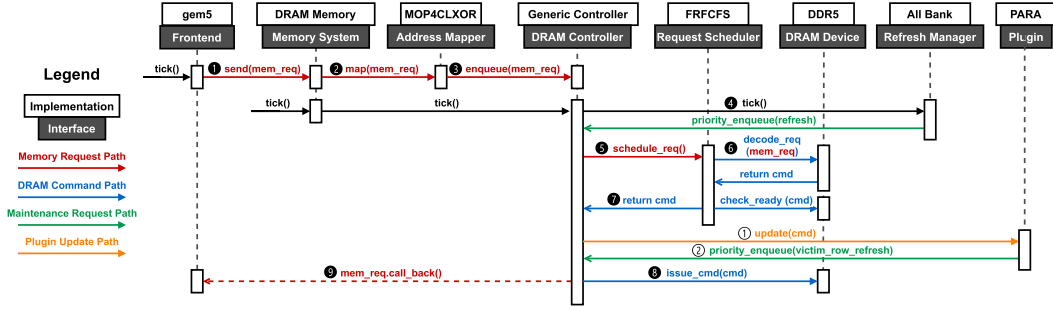


Fig. 1. High-level software architecture of Ramulator 2.0 using an example DDR5 system configuration.

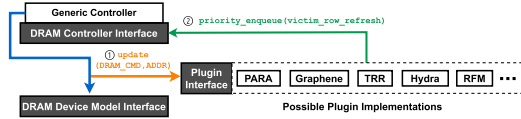


Fig. 2. Implementing RowHammer mitigation techniques as controller plugins. Legend is in Fig. 1.

mitigation implementation then updates its internal state (e.g., generates a random number for PARA, updates the row activation count table for Graphene, or queries the row count cache for Hydra). If the implementation detects the need to refresh the potential RowHammer victim rows, it calls the `priority_enqueue()` function (② in Figs. 1 and 2) of the memory controller interface to send a high-priority refresh request for the identified victim rows, ready to be scheduled in the following cycles, as determined by the mitigation techniques. To showcase the modularity and extensibility of memory controller plugins, Section III-C provides a cross-sectional evaluation of the performance overhead of six different RowHammer mitigation techniques, all implemented as memory controller plugins.

B. Concise and Intuitive DRAM Specifications

Ramulator 2.0 facilitates easy modification and extension of DRAM specifications (e.g., the organization of the DRAM device hierarchy, DRAM commands, timing constraints, mapping between DRAM commands and organization levels) in two major ways. First, Ramulator 2.0 allows the user to directly define the DRAM specifications *by their names* with human-readable string literals, as Listing 1 shows.

```

1 // Different levels in the organization hierarchy
2 inline static constexpr ImplDef m_levels = {
3   "channel", "rank", "bankgroup",
4   "bank", "row", "column",
5 };
6 // Different DRAM commands
7 inline static constexpr ImplDef m_commands = {
8   "ACT", "PRE", "PREab", "RD", "WR", "REF"
9 };
10 // Mapping between commands and levels
11 inline static const ImplLUT m_cmd_scopes = LUT (
12   m_commands, m_levels, {
13     {"ACT", "row"}, {"PRE", "bank"},
14     {"RD", "column"}, {"WR", "column"},
15     {"REF", "rank"}, {"PREab", "rank"},
16   }
17 );

```

list. 1. Example definition of DRAM organization and commands.

Internally, Ramulator 2.0 *automatically* encodes these string literals into integers. These integers are used to *efficiently* index the the lookup table-based finite state machines that Ramulator 2.0 uses to model the hierarchical organization and behavior

of DRAM devices, similarly to Ramulator 1.0 [2]. This encoding is done statically at *compile-time* within the frequently queried and updated DRAM device model so that it does *not* incur any run-time performance overhead (e.g., the expression `m_levels["bank"]` is a `constexpr` expression that is evaluated to the integer “3” by the compiler). Other components in the simulated system that need to know the DRAM device’s specifications (e.g., the organization of the DRAM device hierarchy, DRAM commands, timing constraints, the mapping between DRAM commands and organization levels) can query the DRAM specification with string literals during initialization to get the underlying integer encoding (or an error indicating the component is incompatible with the DRAM specification). Doing so *completely* decouples the DRAM specifications from other parts of the simulated system, thereby achieving higher modularity and extensibility than Ramulator 1.0.

Based on these string-literal based definitions, Ramulator 2.0 develops a concise and human-readable way to model the timing constraints of DRAM commands. The key idea is to define timing constraints based on the permutation of the preceding and following DRAM commands. Doing so reduces redundant code by merging the timing constraint definitions that have the same numerical value but are between different pairs of preceding and following DRAM commands into a single definition. For example, Listing 2 shows the definition of the timing constraint `nRCD` that specifies the minimum delay between a preceding `ACT` command and either a following `RD` or `WR` command at the bank level. With this modeling, Ramulator 2.0 defines the key DDR4 timing constraints with only 32 lines of code, a 61% reduction from Ramulator 1.0’s 82 lines. Such code deduplication enables the addition of new DRAM standards in an easier and less error-prone way.

```

1 { .level = "bank",
2   .preceding = {"ACT"}, .following = {"RD", "WR"},
3   .latency = V("nRCD") },

```

list. 2. Example definition of timing constraints.

Second, Ramulator 2.0 implements the DRAM commands (e.g., the state changes caused by the DRAM commands and the prerequisite commands based on the current state) using a library of lambda functions. These functions are implemented in a templated way so that they are defined only once, but can be *reused* many times for similar DRAM commands across *different* standards. As an example, Listing 3 shows a part of implementations of the `RFMab` command (all-bank refresh management, which exists in the DDR5 [7], LPDDR5 [8], GDDR6 [9], and HBM3 [10] standards) that requires all the banks in the DRAM device to be closed before it can be issued.


```

1  template <class DRAM_t>
2  int RequireAllBanksClosed(typename DRAM_t::Node* node,
3  int cmd, int target_id, Clk_t clk) {
4  // for all banks {
5  // ...
6  if (bank->m_state == DRAM_t::m_states["Closed"]) {
7  continue;
8  } else {
9  return T::m_commands["PREab"];
10 }
11 // }
12 return cmd;
13 };
14 // In DDR5.cpp
15 m_pregs[m_levels["rank"]][m_commands["RFMab"]] =
16   RequireAllBanksClosed<DDR5>;
17 // In LPDDR5.cpp
18 m_pregs[m_levels["rank"]][m_commands["RFMab"]] =
19   RequireAllBanksClosed<LPDDR5>;
20 // In GDDR6.cpp
21 m_pregs[m_levels["channel1"]][m_commands["RFMab"]] =
22   RequireAllBanksClosed<GDDR6>;
23 // In HBM3.cpp
24 m_pregs[m_levels["channel1"]][m_commands["RFMab"]] =
25   RequireAllBanksClosed<HBM3>;

```

list. 3. Example Implementation of a DRAM Command, *RFMab* (shared across different DRAM standards, including DDR5, LPDDR5, GDDR6, HBM3).

In the lambda function library source file (i.e., `lambda/preg.h`), Ramulator 2.0 defines a generic function `RequireAllBanksClosed(...)` that checks for all banks in the organization hierarchy if all of them are closed (lines 5-6). If so, it simply returns the input command argument `cmd` (line 11), indicating that no prerequisite command is needed for `cmd`. Otherwise, it returns the `PREab` (precharge all-bank) command to close all the banks first. This function is templated on the DRAM standard implementation (i.e., the `DRAM_t` template parameter on line 1) so that it can automatically get the correct integer encoding of the commands and states for different DRAM standards at compile-time. By reusing this templated function in different DRAM standards (lines 15, 17, 19, and 21), implementing the prerequisite checks for the *RFMab* command needs only a single line of code in each standard (instead of duplicating the entire `RequireAllBanksClosed` function for each DRAM standard as in Ramulator 1.0).

III. VALIDATION AND EVALUATION

A. Validating the Correctness of Ramulator 2.0

To make sure Ramulator 2.0's memory controller and DRAM device model implementation is correct (i.e., the DRAM commands issued by the controller obey both the timing constraints and the state transition rules), we verify the DRAM command trace against Micron's DDR4 Verilog Model [22] using a similar methodology to prior works [2], [3], [4]. To do so, we implement a DRAM command trace recorder as a DRAM controller plugin that can store the issued DRAM commands with the addresses and time stamps using the DDR4 Verilog Model's format. We collect DRAM command traces from eight streaming-access and eight random-access synthetic memory traces and different intensities (i.e., the number of non-memory instructions between memory instructions). We feed the DRAM command trace to the Verilog Model, configured to use the same DRAM organization and timings as we use in Ramulator 2.0. We find no timing or state transition violations.

B. Performance of Ramulator 2.0

We compare the simulation speed of Ramulator 2.0 with three other cycle-accurate DRAM simulators: Ramulator 1.0 [2], DRAMsim2 [3], DRAMsim3 [4], and USIMM [1]. All four simulators are configured with comparable system parameters. We generate two memory traces, one with a random access pattern

TABLE I
SIMULATION PERFORMANCE COMPARISON

Simulator (gcc-12 -O3)	Runtime (sec) min/avg/max.		Avg. Requests/sec	
	Random	Stream	Random	Stream
Ramulator 2.0	50.3/50.6/51.4	26.1/26.2/26.4	98.8K	190.8K
Ramulator 1.0	58.2/59.0/62.3	31.7/31.9/33.0	84.7K	156.7K
DRAMsim3	51.4/51.7/52.3	37.5/37.8/38.6	96.7K	132.3K
DRAMsim2	51.6/51.9/52.4	53.7/53.9/54.1	96.3K	92.8K
USIMM	402.9/407.0/410.0	31.2/31.3/31.4	12.3K	159.7K

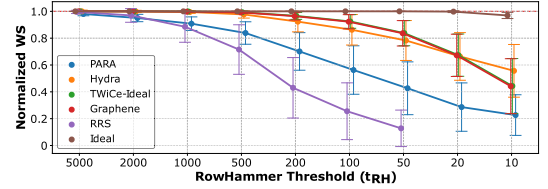


Fig. 3. Performance overhead of RowHammer mitigation techniques versus different RowHammer thresholds.

and another with a streaming access pattern, each containing five million memory requests (read-write ratio = 4:1). For each simulator and trace, we repeat the simulation for each trace ten times. Table I shows the minimum, average, and maximum simulation runtimes across the ten repetitions. We conclude that, despite the increased modularity and extensibility, Ramulator 2.0 achieves a comparably fast (and even faster) simulation speed versus other existing cycle-accurate DRAM simulators. We provide the complete set of scripts, configurations, and traces to reproduce our results in [6].

C. Cross-Sectional Study of RowHammer Mitigations

To demonstrate the modularity and extensibility of Ramulator 2.0, we implement six different RowHammer mitigation techniques, PARA [12], an idealized version of TWiCe [13], Graphene [14], Hydra [15], Randomized Row-Swap (RRS) [16], and an ideal refresh-based mitigation (Ideal) [17]. All of these mechanisms are implemented in the form of memory controller plugins as described in Section II-A1. Fig. 3 shows the performance overhead (weighted speedup normalized to a baseline configuration running the same workloads *without* any RowHammer mitigation, y-axis) of different RowHammer mitigations as the RowHammer threshold (i.e., the minimum number of DRAM row activations to cause at least one bitflip, t_{RH} , x-axis) decreases from 5000 to 10. We use traces generated from SPEC2006 [23] and SPEC2017 [24] to form 25 four-core multiprogrammed workloads that we feed through a simplistic out-of-order core model (the complete set of scripts and traces to reproduce these experiments are in [6]).

We make the following two observations. First, all evaluated RowHammer mitigations (except for Ideal) cause significant performance overhead compared to the ideal mitigation as t_{RH} decreases to very low values. Second, for $t_{RH} < 50$, the performance overhead of RRS becomes too high for the simulation to make progress. The reason for this is that the activation caused by a row swap triggers even more row swaps, preventing DRAM from serving memory access requests. We conclude that existing RowHammer mitigation techniques are not scalable enough to very low t_{RH} values (below 50). As such, more research effort is needed to develop more efficient and scalable RowHammer mitigation techniques.

IV. CONCLUSION

We present Ramulator 2.0, a modern, modular, and extensible DRAM simulator as a successor to Ramulator 1.0. We introduce the key design features of Ramulator 2.0 and demonstrate its high modularity, extensibility, and performance. We hope that Ramulator 2.0's modular and extensible software architecture and concise and intuitive modeling of DRAM facilitates more agile memory systems research.

REFERENCES

- [1] N. Chatterjee et al., "USIMM: The Utah Simulated memory module," University of Utah, Tech. Rep. UUCS-12-002, 2012.
- [2] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible DRAM simulator," *IEEE Comput. Archit. Lett.*, vol. 15, no. 1, pp. 45–49, Jan.–Jun. 2016.
- [3] P. Rosenfeld, E. Cooper-Balis, and B. Jacob, "DRAMSim2: A cycle accurate memory system simulator," *IEEE Comput. Archit. Lett.*, vol. 10, no. 1, pp. 16–19, Jan./Jun. 2011.
- [4] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator," *IEEE Comput. Archit. Lett.*, vol. 19, no. 2, pp. 106–109, Jul./Dec. 2020.
- [5] JESD79–4C: DDR4 SDRAM Standard, JEDEC, Arlington County, VA, USA, 2020.
- [6] SAFARI Research Group, "Ramulator2 GitHub repository," 2023. [Online]. Available: <https://github.com/CMU-SAFARI/ramulator2>
- [7] JESD79–5: DDR5 SDRAM Standard, JEDEC, Arlington County, VA, USA, 2020.
- [8] JESD209–5A: LPDDR5 SDRAM Standard, JEDEC, Arlington County, VA, USA, 2020.
- [9] JESD250C: Graphics Double Data Rate 6 (GDDR6) Standard, JEDEC, Arlington County, VA, USA, 2021.
- [10] JESD238A: High Bandwidth Memory (HBM3) DRAM, JEDEC, Arlington County, VA, USA, 2023.
- [11] ISO, ISO/IEC 14882:2020 Programming languages — C++, ISO/IEC JTC 1/SC 22, ICS: 35.060, Ed. 6.
- [12] Y. Kim et al., "Flipping bits in memory without accessing them: An experimental study of dram disturbance errors," in *Proc. ACM/IEEE 41st Int. Symp. Comput. Architecture*, 2014, pp. 361–372.
- [13] E. Lee, I. Kang, S. Lee, G. E. Suh, and J. H. Ahn, "TWiCe: Preventing row-hammering by exploiting time window counters," in *Proc. ACM/IEEE 46th Int. Symp. Comput. Architecture*, 2019, pp. 385–396.
- [14] Y. Park, W. Kwon, E. Lee, T. J. Ham, J. Ho Ahn, and J. W. Lee, "Graphene: Strong yet lightweight row hammer protection," in *Proc. IEEE Int. Symp. Microarchitecture*, 2020, pp. 1–13.
- [15] D.-H. Kim, P. J. Nair, and M. K. Qureshi, "Hydra: Enabling low-overhead mitigation of row-hammer at ultra-low thresholds via hybrid tracking," in *Proc. ACM/IEEE Int. Symp. Comput. Architecture*, 2022, pp. 699–710.
- [16] G. Saileshwar et al., "Randomized row-swap: Mitigating row hammer by breaking spatial correlation between aggressor and victim rows," in *Proc. Int. Conf. Architect. Support Program. Lang. Operating Syst.*, 2022, pp. 1056–1069.
- [17] J. S. Kim et al., "Revisiting RowHammer: An experimental analysis of modern devices and mitigation techniques," in *Proc. ACM/IEEE Annu. Int. Symp. Comput. Architecture*, 2020, pp. 638–651.
- [18] J. Lowe-Power et al., "The gem5 simulator: Version 20.0+," 2020, *arXiv: 2007.03152*.
- [19] D. Sanchez and C. Kozyrakis, "ZSim: Fast and accurate microarchitectural simulation of thousand-core systems," in *Proc. ACM Annu. Int. Symp. Comput. Architecture*, 2013, pp. 475–486.
- [20] JESD79–3: DDR3 SDRAM Standard, Arlington County, VA, USA, 2012.
- [21] JESD235D: High Bandwidth Memory DRAM (HBM1, HBM2), JEDEC, Arlington County, VA, USA, 2021.
- [22] Micron Technology, "Micron DDR4 verilog model," 2018. [Online]. Available: https://media-www.micron.com/-/media/client/global/documents/products/sim-model/dram/ddr4/ddr4_verilog_models.zip
- [23] Standard Performance Evaluation Corp., "SPEC CPU 2006," 2006. [Online]. Available: <http://www.spec.org/cpu2006/>
- [24] Standard Performance Evaluation Corp., "SPEC CPU 2017," 2017. [Online]. Available: <http://www.spec.org/cpu2017/>